

DSA0318-Natural Language Processing using python

List of Lab Program

1. Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.
2. Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.
3. Write program demonstrates how to perform morphological analysis using the NLTK library in Python.
4. Implement a finite-state machine for morphological parsing. In this example, we'll create a simple machine to generate plural forms of English nouns using python.
5. Use the Porter Stemmer algorithm to perform word stemming on a list of words using python libraries.
6. Implement a basic N-gram model for text generation. For example, generate text using a bigram model using python.
7. Write program using the NLTK library to perform part-of-speech tagging on a text.
8. Implement a simple stochastic part-of-speech tagging algorithm using a basic probabilistic model to assign POS tags using python.
9. Implement a rule-based part-of-speech tagging system using regular expressions using python.
10. Implement transformation-based tagging using a set of transformation rules, apply a simple rule to tag words using python.
11. Implement a simple top-down parser for context-free grammars using python.
12. Implement an Earley parser for context-free grammars using a simple python program.
13. Generate a parse tree for a given sentence using a context-free grammar using python program.
14. Create a program in python to check for agreement in sentences based on a context-free grammar's rules.
15. Implement probabilistic context-free grammar parsing for a sentence using python.
16. Implement a Python program using the SpaCy library to perform Named Entity Recognition (NER) on a given text.
17. Write program demonstrates how to access WordNet, a lexical database, to retrieve synsets and explore word meanings in python.
18. Implement a simple FOPC parser for basic logical expressions using python program.
19. Create a program for word sense disambiguation using the Lesk algorithm using python.
20. Implement a basic information retrieval system using TF-IDF (Term Frequency-Inverse Document Frequency) for document ranking using python.
21. Create a python program that performs syntax-driven semantic analysis by extracting noun phrases and their meanings from a sentence.
22. Create a python program that performs reference resolution within a text.
23. Develop a python program that evaluates the coherence of a given text.
24. Create a python program that recognizes dialog acts in a given dialog or conversation.
25. Utilize the GPT-3 model to generate text based on a given prompt. Make sure to install the OpenAI GPT-3 library in python implementation.
26. Implement a machine translation program using the Hugging Face Transformers library, translate English text to French using python.

1. Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.

```
import re

text = "My email is test@example.com and phone is 9876543210"

# Match email pattern
email_pattern = r'\b[\w.-]+@[ \w.-]+\.\w+\b'
emails = re.findall(email_pattern, text)

# Search phone number
phone_pattern = r'\b\d{10}\b'
phone = re.search(phone_pattern, text)

print("Emails found:", emails)
print("Phone found:", phone.group() if phone else "Not found")
```

2. Implement a basic finite state automaton that recognizes strings ending with 'ab'.

```
def fsa_ends_with_ab(string):
    state = 0
    for char in string:
        if state == 0:
            state = 1 if char == 'a' else 0
        elif state == 1:
            state = 2 if char == 'b' else (1 if char == 'a' else 0)
        elif state == 2:
            state = 1 if char == 'a' else 0
    return state == 2

print(fsa_ends_with_ab("aaab")) # True
```

3. Write program demonstrates how to perform morphological analysis using the NLTK library in Python.

```
import nltk
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')

lemmatizer = WordNetLemmatizer()
words = ["running", "better", "cats"]

for word in words:
    print(word, "->", lemmatizer.lemmatize(word))
```

4. Implement a finite-state machine for morphological parsing (Plural generation).

```
def pluralize(noun):
    if noun.endswith('y'):
```

```
        return noun[:-1] + 'ies'
    elif noun.endswith(('s', 'x', 'z', 'ch', 'sh')):
        return noun + 'es'
    else:
        return noun + 's'

print(pluralize("cat"))
print(pluralize("baby"))
```

5. Use the Porter Stemmer algorithm to perform word stemming.

```
from nltk.stem import PorterStemmer

ps = PorterStemmer()
words = ["running", "flies", "happiness"]

for word in words:
    print(word, "->", ps.stem(word))
```

6. Implement a basic Bigram model for text generation.

```
import random
from nltk.util import bigrams

text = "I love natural language processing"
words = text.split()

bi = list(bigrams(words))
model = {}

for w1, w2 in bi:
    model.setdefault(w1, []).append(w2)

word = random.choice(words)
result = [word]

for _ in range(5):
    word = random.choice(model.get(word, words))
    result.append(word)

print("Generated Text:", " ".join(result))
```

7. POS tagging using NLTK.

```
import nltk
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

text = "NLP is very interesting"
tokens = nltk.word_tokenize(text)

print(nltk.pos_tag(tokens))
```

8. Implement a simple stochastic POS tagging algorithm.

```
from collections import defaultdict

training_data = [("dog", "NN"), ("barks", "VB"),
                 ("cat", "NN"), ("runs", "VB")]

prob = defaultdict(lambda: defaultdict(int))

for word, tag in training_data:
    prob[word][tag] += 1

def stochastic_tag(word):
    tags = prob[word]
    return max(tags, key=tags.get) if tags else "NN"

print(stochastic_tag("dog"))
```

9. Rule-based POS tagging using regular expressions.

```
import nltk

patterns = [
    (r'.*ing$', 'VBG'),
    (r'.*ed$', 'VBD'),
    (r'.*s$', 'NNS'),
    (r'.*', 'NN')
]

tagger = nltk.RegexpTagger(patterns)
print(tagger.tag(["running", "played", "cats", "tree"]))
```

10. Implement transformation-based tagging.

```
def transform_tag(word, tag):
    if word.endswith("ing"):
        return "VBG"
    return tag

print(transform_tag("running", "NN"))
```

11. Implement a simple top-down parser for CFG.

```
grammar = {
    "S": [["NP", "VP"]],
    "NP": [["Det", "N"]],
    "VP": [["V", "NP"]],
    "Det": [["the"]],
    "N": [["cat"], ["dog"]],
    "V": [["chased"]]
}

def parse(symbol, words):
    if symbol not in grammar:
```

```
        return words[0] == symbol
    for rule in grammar[symbol]:
        if all(parse(r, [w]) for r, w in zip(rule, words)):
            return True
    return False

print(parse("NP", ["the", "cat"]))
```

12. Implement an Earley parser for CFG.

```
import nltk

grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> 'she'
VP -> 'runs'
""")

parser = nltk.EarleyChartParser(grammar)
sentence = "she runs".split()

for tree in parser.parse(sentence):
    print(tree)
```

13. Generate a parse tree for a sentence using CFG.

```
import nltk

grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> 'I'
VP -> 'run'
""")

parser = nltk.ChartParser(grammar)

for tree in parser.parse("I run".split()):
    tree.pretty_print()
```

14. Program to check agreement in sentences.

```
def check_agreement(subject, verb):
    if subject == "he" and verb.endswith("s"):
        return True
    return False

print(check_agreement("he", "runs"))
```

15. Implement PCFG parsing.

```
import nltk
```

```
grammar = nltk.PCFG.fromstring("""
S -> NP VP [1.0]
NP -> 'she' [1.0]
VP -> 'runs' [1.0]
""")

parser = nltk.ViterbiParser(grammar)

for tree in parser.parse("she runs".split()):
    print(tree)
```

16. Named Entity Recognition using SpaCy.

```
import spacy
nlp = spacy.load("en_core_web_sm")

doc = nlp("Apple is opening a new office in Chennai.")

for ent in doc.ents:
    print(ent.text, ent.label_)
```

17. Access WordNet to retrieve synsets.

```
from nltk.corpus import wordnet
import nltk
nltk.download('wordnet')

synsets = wordnet.synsets("bank")

for syn in synsets:
    print(syn.name(), "-", syn.definition())
```

18. Simple FOPC parser.

```
def parse_fopc(expr):
    if "(" in expr and ")" in expr:
        predicate = expr.split("(")[0]
        args = expr[expr.find("(")+1:expr.find(")"]].split(",")
        return predicate, args
    return None

print(parse_fopc("Loves(John,Mary)"))
```

19. Word Sense Disambiguation using Lesk algorithm.

```
from nltk.wsd import lesk

sentence = "I went to the bank to deposit money".split()
sense = lesk(sentence, "bank")

print(sense, "-", sense.definition())
```

20. TF-IDF based Information Retrieval.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = ["NLP is fun", "I love machine learning"]
vectorizer = TfidfVectorizer()
tfidf = vectorizer.fit_transform(docs)

query = vectorizer.transform(["NLP"])
scores = cosine_similarity(query, tfidf)

print(scores)
```

21. Syntax-driven semantic analysis (Extract noun phrases).

```
import nltk

text = "The cat sits on the mat"
tokens = nltk.word_tokenize(text)
pos = nltk.pos_tag(tokens)

noun_phrases = [word for word, tag in pos if tag.startswith('NN')]
print(noun_phrases)
```

22. Reference resolution program.

```
def resolve_reference(text):
    sentences = text.split(".")
    if len(sentences) > 1 and "He" in sentences[1]:
        return sentences[0].split()[0]
    return None

print(resolve_reference("John went home. He slept."))
```

23. Evaluate coherence of text.

```
from nltk.tokenize import sent_tokenize

def coherence_score(text):
    sentences = sent_tokenize(text)
    return len(sentences)

print(coherence_score("This is a sentence. This is another one."))
```

24. Recognize dialog acts.

```
def dialog_act(sentence):
    if sentence.endswith("?"):
        return "Question"
    elif sentence.lower().startswith("please"):
        return "Request"
```

```
        else:
            return "Statement"

print(dialog_act("How are you?"))
```

25. Utilize GPT-3 model to generate text.

```
import openai

openai.api_key = "YOUR_API_KEY"

response = openai.Completion.create(
    model="text-davinci-003",
    prompt="Explain Natural Language Processing",
    max_tokens=50
)

print(response.choices[0].text.strip())
```

26. Machine Translation using Hugging Face Transformers.

```
from transformers import pipeline

translator = pipeline("translation_en_to_fr")

result = translator("Natural Language Processing is amazing.")
print(result[0]['translation_text'])
```